Revisión Sistemática de la Literatura sobre la Clasificación de Requerimientos No Funcionales con Desbalance de Clases: Aumentación de Datos, BERT y Modelos de Lenguaje de Gran Escala

Andres Lino Montes Mamani, Cristian Cabana Sulca, Alessandro Pastor Mamani Mamani
Escuela Profesional de Ingeniería de Sistemas
Universidad Peruana Unión, Juliaca, Perú
Asesor: Mg. Esteban Tocto Cano

***Resumen*—Contexto. La clasificación automática de requerimientos no funcionales (NFR) está limitada por el desbalance extremo de clases. La literatura ofrece técnicas de aumentación de datos, clasificadores basados en BERT y modelos de lenguaje de gran escala (LLM), pero no los compara de forma controlada. Objetivo. Identificar, evaluar y sintetizar la evidencia sobre las técnicas contra el desbalance en la clasificación de NFR, los clasificadores con que se combinan, el uso de LLM como clasificadores, las fuentes de datos y el rigor de la evaluación. Método. Revisión sistemática según Kitchenham y Charters y PRISMA 2020, con cuatro cadenas de búsqueda en Scopus (2017–2026), criterios explícitos de selección, validación del cribado por dos revisores, mapeo de los estudios incluidos y extracción de datos y evaluación de calidad del texto completo. Resultados. De 565 registros se incluyeron 145 estudios primarios; el mapeo abarca 136 y la síntesis del texto completo, 58. Desde 2024 la investigación se desplazó a los transformers y los LLM. Solo 44 estudios mencionan el desbalance en su resumen; casi todos los que lo tratan evalúan una sola técnica, y los dos que comparan varias aumentaciones lo hacen en la tarea binaria de seguridad o con un clasificador recurrente. En el texto completo, 17 de 58 estudios reportan F1 macro, 10 controlan los duplicados entre entrenamiento y prueba y 19 aplican pruebas estadísticas. Ningún estudio compara TAOS, EDA y SMOTE-Tomek con BERT en la clasificación multiclase de NFR ni contrasta el resultado con LLM. Conclusiones. La brecha es metodológica además de temática: la comparación pendiente debe hacerse sobre conjuntos sin duplicados, con métricas por clase y con pruebas de significancia.**

***Index Terms*—revisión sistemática, requerimientos no funcionales, desbalance de clases, aumentación de datos, BERT, modelos de lenguaje de gran escala**

I. Introducción

Los requerimientos no funcionales (NFR) expresan atributos de calidad como la seguridad, el desempeño, la portabilidad o la tolerancia a fallos, y su clasificación automática a partir de las especificaciones de software es una tarea consolidada en la ingeniería de requerimientos [1]–[3]. El principal obstáculo es el desbalance de clases: en la partición de entrenamiento que usa TAOS [4], la clase funcional tiene 557 requerimientos y portabilidad, solo dos.

El estado del arte asociado a esta investigación mostró que la aumentación de datos, los clasificadores basados en BERT y los LLM se han estudiado por separado. Aquel análisis se basó en los resúmenes. Esta revisión sistemática valida la selección con dos revisores, evalúa la calidad de cada estudio y extrae los datos del texto completo, de modo que las brechas queden respaldadas por evidencia verificable. Además, examina con qué rigor se evalúan los estudios, porque una técnica de aumentación solo puede compararse con otra si el protocolo evita la fuga de datos.

La Sección II describe el protocolo, la III presenta los resultados por pregunta de investigación, la IV los discute, la V analiza las amenazas a la validez y la VI concluye.

II. Protocolo de la revisión

La revisión sigue las guías para revisiones sistemáticas en ingeniería de software de Kitchenham y Charters [5], [6] y reporta el proceso conforme a la declaración PRISMA 2020 [7].

II-A. Preguntas de investigación

La Tabla I delimita el alcance con el esquema PICOC y la Tabla II formula las preguntas de investigación.

II-B. Fuente y cadenas de búsqueda

La búsqueda se realizó en Scopus, la mayor base de resúmenes y citas revisada por pares, que indexa las editoriales en que se publica la investigación en ingeniería de requerimientos: IEEE, ACM, Springer, Elsevier y MDPI. De los 145 estudios incluidos, 53 son de IEEE, 34 de Springer, 14 de Elsevier, 10 de MDPI y 5 de ACM, entre ellos las actas de la conferencia RE y de sus talleres, la revista *Requirements Engineering* y la revista *Information and Software Technology*. Se descartó una búsqueda en Web of Science porque la institución no tiene suscripción activa. Los textos completos se obtuvieron de las versiones de acceso abierto y, para los artículos de Elsevier, de ScienceDirect mediante el acceso institucional.

Se aplicaron cuatro cadenas (Tabla III). Todas comparten un bloque común que combina los términos referidos a requerimientos no funcionales, de calidad o de software con el concepto de clasificación, y cada cadena añade uno de los

Tabla I
ALCANCE DE LA REVISIÓN SEGÚN EL ESQUEMA PICOC

Elemento	Definición
Población	Estudios que clasifican automáticamente requerimientos de software escritos en lenguaje natural en tipos de requerimiento o atributos de calidad.
Intervención	Técnicas contra el desbalance y la escasez de datos (remuestreo tipo SMOTE y SMOTE-Tomek, aumentación léxica tipo EDA, TextAttack y TAOS, retrotraducción, generación con LLM, funciones de pérdida ponderadas) y clasificadores basados en transformers o en LLM.
Comparación	Clasificadores entrenados sin tratamiento del desbalance, aprendizaje automático clásico y transformers ajustados sin aumentación.
Resultados	F1 macro, F1 ponderado, F1 por clase y MCC; desempeño en las clases minoritarias; rigor del protocolo de evaluación.
Contexto	Ingeniería de requerimientos; conjuntos como PROMISE, PROMISE_exp, PURE y SecReq, y texto de usuarios o de proyectos de código abierto.

Tabla II
PREGUNTAS DE INVESTIGACIÓN

ID	Pregunta y motivación
PR1	¿Qué enfoques de clasificación se aplican a los NFR y cómo han evolucionado? *Sitúa el desempeño de BERT y de los LLM frente a los métodos previos.*
PR2	¿Qué técnicas se usan contra el desbalance y la escasez de datos, con qué clasificadores y con qué efecto en las clases minoritarias? *Núcleo de la comparación entre TAOS, EDA y SMOTE-Tomek.*
PR3	¿Cómo se usan los LLM como clasificadores de requerimientos y frente a qué se comparan? *Fundamenta la comparación con BERT aumentado.*
PR4	¿Qué conjuntos y fuentes de datos se emplean y cómo se amplían? *Respalda la ampliación con requerimientos de GitHub.*
PR5	¿Con qué rigor se evalúan los estudios (partición, fuga de datos, métricas por clase, pruebas estadísticas, reproducibilidad)? *Determina la confianza que merece la evidencia.*

Tabla III
REGISTROS RECUPERADOS EN SCOPUS POR CADENA DE BÚSQUEDA

ID	Eje temático	Registros
S1	Aumentación de datos y desbalance	84
S2	BERT y transformers	214
S3	LLM como clasificador	173
S4	GitHub, auto-etiquetado y ampliación de datos	94
	Total (con duplicados entre cadenas)	565
	Registros únicos	434

ejes de la revisión. El operador de proximidad `W/3` evita siglas ambiguas como SRS o EDA. Los filtros son el periodo 2017–2026, los idiomas inglés y español, y los tipos artículo, ponencia, revisión y capítulo de libro. Las cadenas completas figuran en el Anexo A. La búsqueda se ejecutó en octubre de 2026.

Tabla IV
CRITERIOS DE INCLUSIÓN (CI) Y EXCLUSIÓN (CE)

Código	Criterio
CI1	Estudio primario que clasifica automáticamente texto de requerimientos en tipos de requerimiento (FR/NFR o subclases de NFR) o en atributos de calidad.
CI2	Usa aprendizaje automático, aprendizaje profundo, transformers o LLM.
CI3	Reporta una evaluación empírica cuantitativa.
CI4	Publicado entre 2017 y 2026, en inglés o español.
CE1	Fuera del dominio de software: *requirement* o *quality* se usan con otro sentido.
CE2	Otra tarea de ingeniería de requerimientos o de software (ambigüedad, trazabilidad, generación, priorización, riesgo, bugs), o clases que no son tipos de requerimiento.
CE3	Sin evaluación empírica.
CE4	Estudio secundario: se usa como contexto, no en la síntesis.
CE5	Duplicado, o versión resumida o extendida de otro registro.

II-C. Criterios de selección

La Tabla IV define los criterios. Un estudio se incluye si cumple los cuatro criterios de inclusión (CI) y se excluye si cumple cualquiera de exclusión (CE). El texto de usuarios (reseñas, solicitudes, issues) solo se incluye cuando las clases son tipos de requerimiento o atributos de calidad. Cuando el título y el resumen no bastan para decidir, el estudio pasa a la lectura del texto completo.

II-D. Proceso de selección y fiabilidad

La selección tiene tres etapas: eliminación de duplicados entre cadenas, cribado por título y resumen, y evaluación de elegibilidad con el texto completo (Fig. 1). El cribado inicial se realizó con apoyo de un modelo de lenguaje, y cada decisión se documentó junto con el criterio aplicado. Para evaluar su fiabilidad, dos autores revisaron de forma independiente y ciega una muestra formada por todos los estudios incluidos y el 20 % de los excluidos. Cada uno trabajó con un orden aleatorio distinto y sin conocer las decisiones del cribado inicial ni las del otro revisor. El acuerdo entre los tres evaluadores, medido con el kappa de Fleiss, fue κ = [por completar], y el kappa de Cohen entre pares osciló en [por completar]. Los desacuerdos se resolvieron por mayoría y discusión.

Los 565 registros se redujeron a 434 tras eliminar 131 duplicados entre cadenas. El cribado por título y resumen excluyó 277 (142 por CE1, 129 por CE2, 4 por CE3 y 2 por CE5), identificó 12 estudios secundarios y retuvo 145 estudios primarios. De estos se obtuvo el texto completo de 67: de las versiones de acceso abierto de las editoriales y, cuando no existían, de los preprints de arXiv de los mismos autores. La lectura excluyó 9: seis por CE2, porque sus clases no son tipos de requerimiento (defectos de redacción [8], etiquetas de taxonomías de dominio o de privacidad para la trazabilidad en tres estudios, la utilidad de las reseñas y las categorías de Kano); dos por CE1, porque clasifican requerimientos de sistemas aeroespaciales y no de software; y uno por CE3, la demostración de una herramienta sin evaluación nueva.

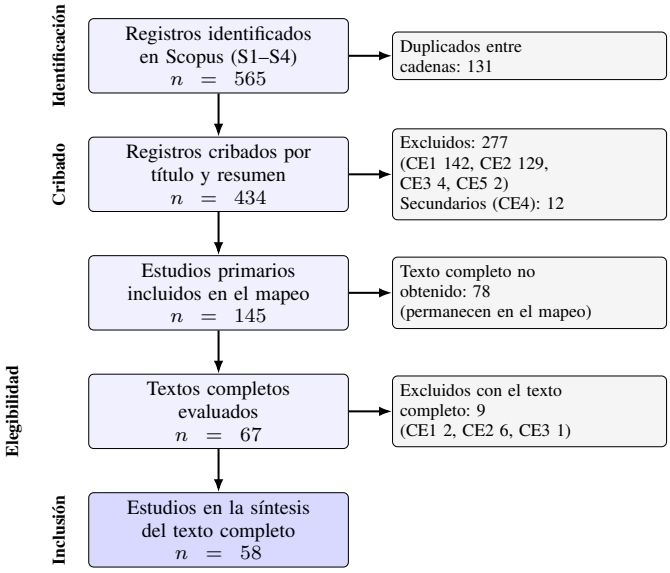


Fig. 1. Diagrama de flujo PRISMA 2020 de la selección de estudios.

II-E. Mapeo y síntesis del texto completo

La revisión tiene dos niveles. El *mapeo* caracteriza los 136 estudios primarios elegibles a partir del título, el resumen y las palabras clave: enfoque principal, técnica contra el desbalance, fuente de datos, conjuntos y tipo de tarea. Las etiquetas se asignaron por palabras clave con verificación manual. La *síntesis* usa el texto completo de los 58 estudios elegibles disponibles. Los datos se extrajeron con un formulario estandarizado de 23 campos, agrupados en cinco bloques: datos y tarea (fuente del texto, conjunto, idioma, número de requerimientos y de clases, distribución de clases), tratamiento del desbalance (técnica y si se aplica solo al entrenamiento), clasificación (clasificadores y papel del LLM), evaluación (protocolo, control de duplicados, métricas, F1 macro, ponderado y de las clases minoritarias, línea base y pruebas estadísticas) y reproducibilidad (disponibilidad de código y datos, y limitaciones declaradas). Los estudios difieren en conjuntos, clases, particiones y métricas, por lo que no procede un metaanálisis. La síntesis es narrativa y tabular por pregunta de investigación, y la evidencia se pondera según la puntuación de calidad.

II-F. Evaluación de la calidad

La calidad de cada estudio de la síntesis se evaluó con ocho criterios (Tabla V), adaptados de Dybå y Dingsøyr [9] y de las guías de Kitchenham y Charters [5] al problema de la clasificación con desbalance. Cada criterio se puntúa con 1 (sí), 0.5 (parcialmente) o 0 (no), lo que da un total entre 0 y 8. La puntuación no se usó para excluir estudios, sino para ponderar la evidencia. La extracción y la evaluación las realizó un revisor con apoyo de un modelo de lenguaje, verificando cada cifra en el texto del estudio; [dos autores revisan de forma independiente el 20 % de las evaluaciones].

Tabla V
CRITERIOS DE EVALUACIÓN DE LA CALIDAD

ID	Criterio
QA1	Los objetivos o las preguntas de investigación están claramente definidos.
QA2	El conjunto de datos está descrito: origen, tamaño, clases y su distribución.
QA3	El protocolo de evaluación está descrito y evita la fuga de datos: la aumentación se aplica solo al entrenamiento y no hay duplicados entre entrenamiento y prueba.
QA4	Usa métricas adecuadas para el desbalance (F1 macro, F1 por clase o MCC).
QA5	Compara contra una línea base o contra el estado del arte.
QA6	Reporta variabilidad o pruebas de significancia estadística.
QA7	Es reproducible: código, datos e hiperparámetros disponibles o descritos.
QA8	Discute limitaciones o amenazas a la validez.

Tabla VI
ESTUDIOS DEL MAPEO POR ENFOQUE PRINCIPAL Y PERIODO ($n = 136$)

Enfoque	2017–2020	2021–2023	2024–2026	Total
ML clásico	7	8	7	22
Aprendizaje profundo	0	6	3	9
Transformers (BERT y variantes)	1	15	48	64
LLM	0	2	39	41
Total	8	31	97	136
Mencionan el desbalance	4	7	33	44

III. RESULTADOS

III-A. Estudios seleccionados

Los 136 estudios del mapeo se concentran en los últimos años: 90 de los 145 incluidos son de 2025 o 2026 (Tabla VI). La Tabla VII resume los 58 estudios de la síntesis. Su puntuación de calidad media es 5.6 sobre 8 (de 2.5 a 8); 26 estudios alcanzan 6 o más y 4 quedan por debajo de 4.

III-B. PR1: enfoques de clasificación

La Tabla VI muestra tres etapas. Hasta 2020 predomina el aprendizaje automático clásico: SVM, Naive Bayes y regresión logística con representaciones TF-IDF [2], [12]. Entre 2021 y 2023 los transformers, sobre todo BERT, pasan a ser el enfoque principal, a partir de NoRBERT [3]. Desde 2024 crecen los LLM, que en 2024–2026 suman 39 estudios frente a 48 de transformers.

El texto completo matiza la ventaja aparente de los modelos más recientes. Los transformers ajustados mantienen el mejor desempeño en las tareas multiclase cuando se entrenan con datos suficientes: Binkhonain y Zhao [19] reimplementaron cuatro enfoques sobre PROMISE_exp y NoRBERT obtuvo los promedios macro y micro más altos. Sin embargo, en sus subclases raras el resultado se invierte: el enfoque jerárquico con SVM superó a NoRBERT en tres clases pequeñas (A, MN y L). Los clasificadores sin entrenamiento supervisado

Tabla VII
ESTUDIOS ANALIZADOS CON EL TEXTO COMPLETO ($n = 58$) (PARTE 1 DE 2)

Estudio	Tarea	Datos	Tratamiento del desbalance	Clasificador	F1 macro	Dup.	Est.	Cód.	QA
[10] (2017)	Bin.	300 reseñas	Ninguna (muestra balanceada)	Semisupervisado (NB, SVM)	–	?	–	–	4.5
[11] (2018)	MC	Solicitudes OSS (3,000)	Palabras clave, aprendizaje activo	SVM, k-NN, NB	–	?	–	–	5.5
[12] (2018)	MC 11	PROMISE	Elimina PO	12 clasificadores ML	0.76[a]	–	–	–	4
[13] (2019)	MC 12	PROMISE	Ninguna	Similitud Word2Vec	–	n/a	–	–	5
[3] (2020)	Bin./NFR	PROMISE	ROS/RUS (entren.)	BERT	–	–	✓	✓	7.5
[14] (2022)	Bin./4 NFR	PROMISE reetiq.	Omite clases raras	BERT + GAT	–	✓	–	–	5.5
[15] (2022)	NFR	PROMISE ampliado	Conjunto balanceado nuevo	BERT + Bi-LSTM	–	?	–	p	6
[16] (2022)	4 NFR	PROMISE	Omite clases raras	BERT-CNN	–	–	–	–	4
[17] (2022)	Bin.	PROMISE	SMOTE previo[b]	8 modelos DL	–	–	✓	–	4.5
[18] (2023)	NFR 5	Reseñas en árabe	EDA (TextAttack)	RNN + LSTM	–	?	–	–	4
[19] (2023)	MC 12	PROMISE_exp	Descomposición (entren.)	SVM jerárquico	0.51	–	–	✓	7
[20] (2023)	Bin.	Turco (4,600)	Ninguna	BERTurk y 19 más	–	?	–	?	4.5
[21] (2023)	Bin./NFR	PROMISE	Agrupa raras en 'otras'	BERT-BiLSTM-CNN	–	–	✓	p	6.5
[22] (2023)	MC 12	PROMISE	Muestras añadidas a mano	XLNet, BERT	–	?	–	–	4.5
[23] (2023)	Bin./NFR	PROMISE, SecReq	Ninguna	Zero-shot (SBERT)	–	n/a	✓	✓	7.5
[24] (2024)	Seg. CIA	1,135 req. seguridad	Ninguna	SVM + SBERT	–	?	–	p	5
[25] (2024)	Bin.	PROMISE_exp	SMOTE-Tomek previo[b]	Ensamble + CatBoost	–	–	–	p	5
[26] (2024)	NFR	PROMISE_exp	Sobremuestreo previo[b]	Bi-LSTM + SVM	–	–	–	–	3.5
[27] (2024)	MC 12	PROMISE	ROS/RUS (entren.)	NLI, ChatGPT	–	?	–	p	6
[28] (2024)	Multiet.	PROMISE, EHR	Ninguna	BM25 + RL	+13 %	?	–	–	4.5
[29] (2025)	MC	Kaggle (976)	Ninguna	Few-shot (redes prot.)	–	✓	–	p	2.5
[30] (2025)	Bin./NFR	PROMISE, SecReq	Ninguna	5 LLM few-shot	0.92–0.94	?	✓	p	7
[31] (2025)	MC 8	10,006 historias	Focal Loss	BERT, XLNet	–	✓	–	✓	6.5
[32] (2025)	Bin.	PROMISE, FR_NFR	Ninguna	BERT	–	?	–	✓	6
[33] (2025)	Seg. bin.	SecReq	Ninguna	ML, BERT con CWE/CVE	–	✓	✓	✓	7.5
[4] (2025)	MC 12	PROMISE deriv.	TAOS (entren.)	BERT	0.89[c]	–	–	✓	6
[34] (2025)	Calidad ISO	1,000 reseñas	Ninguna	Patrones, crowd, ChatGPT	–	n/a	✓	p	7
[35] (2025)	Bin./NFR	PROMISE (1,521)	SMOTE previo[b]	SVM, RF, CNN, BERT	–	–	–	–	3.5
[36] (2025)	Seg. bin./7	803 NFR	Paráfrasis T5	BERT, ANN, SVM	–	?	–	–	4.5
[37] (2025)	Seg. bin.	SecReq	3 aumentaciones (entren.)	BERT, RoBERTa, XLNet	–	?	–	p	6

alcanzan cifras modestas en las clases raras: con similitud semántica, el F1 de portabilidad fue 0.00 y el de tolerancia a fallos, 0.12 [13].

Las variantes más recientes de BERT siguen dos líneas. Una añade capas recurrentes o convolucionales: BERT-CNN, BERT-BiLSTM-CNN, NFRNet y el modelo apilado LSTM-BERT-LSTM [15], [16], [21], [59]. La otra adapta el preentrenamiento al dominio: RequiBERT, preentrenado con especificaciones, obtuvo un F1 medio de 0.85 en las subclases de NFR frente a 0.81–0.82 de NoRBERT [62], y los modelos preentrenados con descripciones de debilidades y vulnerabilidades (CWE y CVE) mejoraron la detección de requerimientos de seguridad entre dominios [33]. Cuando la taxonomía se hace más profunda, el desempeño cae: un transformer jerárquico alcanzó un F1 macro de 0.90, 0.72 y 0.55 en sus tres niveles [55].

III-C. PR2: técnicas contra el desbalance

Solo 44 de los 136 estudios mencionan el desbalance en su resumen, y 27 describen una técnica para tratarlo (Tabla IX). El remuestreo tipo SMOTE es la más frecuente, seguida de la generación con LLM. La aumentación léxica (EDA), TextAttack o TAOS y la retrotraducción aparecen en uno o dos estudios cada una.

En la síntesis, 27 de 58 estudios aplican alguna técnica, y casi todos evalúan una sola frente a no tratar el desbalance. Solo dos comparan varias aumentaciones bajo el mismo protocolo. Petrillo et al. [37] aumentaron el entrenamiento de SecReq con embeddings contextuales, retrotraducción y paráfrasis, y ajustaron cuatro transformers con los mismos hiperparámetros: la primera técnica llevó la exactitud de BERT de 0.89 a 0.94, mientras que RoBERTa resultó sensible a la aumentación. Aldhafer et al. [50] compararon una aumentación adaptativa, que decide cuántas muestras generar según la clase y la longitud, con la aumentación uniforme tradicional,

Tabla VIII
ESTUDIOS ANALIZADOS CON EL TEXTO COMPLETO ($n = 58$) (PARTE 2 DE 2)

Estudio	Tarea	Datos	Tratamiento del desbalance	Clasificador	F1 macro	Dup.	Est.	Cód.	QA
[38] (2025)	FR/NFR	1,880 reseñas	Ninguna	GPT-4o mini	–	?	–	✓	5.5
[39] (2025)	NFR 11	PROMISE, PURE	Generación GPT-4o (entren.)	SVM, NB, BERT	–	✓	–	✓	6
[40] (2025)	FR/NFR	PROMISE	Ninguna	ChatGPT, Gemini	–	?	–	–	5
[41] (2025)	FR/NFR	PROMISE	Ninguna	GPT-4o, LLaMA 3.3	–	n/a	✓	–	5
[42] (2025)	Usab. 3	Reseñas	Ninguna	BERT-BiLSTM-CNN	–	?	–	p	4.5
[43] (2025)	NFR 5	PURE	SMOTE + RUS	ML, CNN, BERT	–	?	✓	–	4.5
[44] (2025)	Bin.	9 fuentes (4,732)	Ninguna (balanceado)	Ensamble de 7 codif.	–	?	–	?	2.5
[45] (2025)	Jerárq.	PROMISE(_exp)	Ninguna	BERT, RoBERTa	0.34–0.55	?	–	✓	6
[46] (2025)	Bin./NFR	PROMISE, SecReq	Omite clases raras	GPT-4o, Llama 3, SBERT	0.66–0.80	✓	✓	✓	8
[47] (2025)	Bin./NFR	PROMISE, PURE	Ejemplos estratificados	7 LLM few-shot	0.60–0.80	?	–	?	5
[48] (2026)	FR 6	PROMISE, propio	Sobremuestreo	Ensamble BERT	–	?	✓	–	6.5
[49] (2026)	NFR 5	PROMISE + clínicos	Ninguna	BERT, SVM	–	?	✓	p	6
[50] (2026)	Bin./4/10	PROMISE, EHR	Aumentación adaptativa (entren.)	BiGRU	+6.7 pts	?	–	?	5.5
[51] (2026)	MC 12	PROMISE(_exp)	Aumentación (entren.)	BERT + RoBERTa	0.84/0.90	?	✓	✓	8
[52] (2026)	Seg. bin.	SecReq	Ninguna	RoBERTa, LoRA	–	?	–	p	5.5
[53] (2026)	4 NFR	PROMISE, PURE	Focal Loss	Codificador de dominio	0.80[d]	✓	–	–	5.5
[54] (2026)	Bin.	PURE (4,661)	Ninguna	CNN, LSTM, GRU	–	?	–	–	4
[55] (2026)	Jerárq.	4 fuentes, sin dup.	Focal Loss	Transformer jerárquico	0.90/0.72/0.55	✓	✓	✓	8
[56] (2026)	Bin./NFR	PROMISE(_exp), SecReq	Ninguna	RAG + LLM	0.89	✓	–	p	6.5
[57] (2026)	MC 8	GitReq (6,302 issues)	Ninguna	4 LLM zero-shot	0.64	✓	–	✓	6.5
[58] (2026)	Jerárq.	PROMISE_exp	Pesos, Focal Loss	ML, DL, BERT	0.66	?	–	–	5
[59] (2026)	Jerárq.	PROMISE_exp, PURE, SRS	Aumentación BERT previa[b]	LSTM-BERT-LSTM	–	p	✓	?	5.5
[60] (2026)	NFR 11	PROMISE_exp (español)	Generación ChatGPT + RUS	RL, NB, SVM	en tablas	?	✓	✓	7.5
[61] (2026)	NFR 9	Sintético (O3DG)	Generación con LLM	ML y DL	–	p	–	p	5
[62] (2026)	Bin./NFR	PROMISE	Elimina PO	BERT de dominio	0.85	?	–	?	5
[63] (2026)	Seg. bin.	SecReq + PROMISE	Ninguna	Aprendizaje activo federado	–	?	✓	✓	7.5
[64] (2026)	Seg. bin.	Reseñas	Ninguna	SBERT, Paraphrase	–	?	✓	✓	7.5
[65] (2026)	Usab. bin.	300 reseñas	Ninguna	GPT-4.1	–	n/a	✓	p	6

Bin.: binaria FR/NFR; MC: multiclase con FR y NFR (número de clases); Multiet.: multietiqueta; Jerárq.: jerárquica; Seg.: seguridad; ROS/RUS: sobremuestreo y submuestreo aleatorios; (entren.): aplicado solo al entrenamiento. Dup.: controla duplicados entre entrenamiento y prueba; Est.: prueba estadística; Cód.: código o datos disponibles; ✓ sí, – no, ? no se indica, n/a no aplica, p parcial. QA: puntuación de calidad sobre 8. [a]F1 calculado con la precisión y la exhaustividad promedio por clase. [b]El remuestreo o la aumentación se aplican antes de partir el conjunto. [c]Con fuga entre entrenamiento y prueba (Sección III-F). [d]En proyectos no vistos; 0.97 con validación cruzada por oración.

Tabla IX
TÉCNICAS CONTRA EL DESBALANCE IDENTIFICADAS EN LOS RESÚMENES DEL MAPEO

Técnica	Estudios
SMOTE / remuestreo	10
Generación con LLM	5
Otra / no especificada	5
EDA / léxica	2
Pérdida / pesos	2
GAN	1
TextAttack / TAOS	1
Retrotraducción	1
Total	27

y mejoraron el F1 macro en 6.7 puntos en las diez clases de PROMISE. Ambas comparaciones dejan fuera el escenario de esta investigación: la primera es binaria y la segunda usa un BiGRU, y ninguna incluye TAOS ni SMOTE-Tomek.

Entre los estudios de una sola técnica, TAOS [4] combina siete aumentadores de TextAttack con un filtro de paráfrasis y eleva el F1 macro de BERT de 0.44 a 0.89. Pham et al. [39] generaron NFR sintéticos con GPT-4o, pero al probar en PROMISE el modelo entrenado solo con datos reales (F1 ponderado 0.80) superó al combinado (0.77) y al sintético (0.51); el sintético solo ayudó al probar en PURE. Kaur y Kaur [59] aplicaron aumentación contextual con BERT a las subclases minoritarias y la exactitud subió entre 0.2 y 3.76 puntos, aunque el desbalance persistió. La aumentación léxica tipo EDA solo aparece en reseñas en árabe [18], y SMOTE-Tomek, en la tarea binaria y aplicado antes de partir el conjunto [25]. En español, la generación de NFR con ChatGPT combinada con submuestreo mejoró portabilidad y tolerancia a fallos, pero bajó el desempeño global [60]. Said et al. [51] partieron el conjunto antes de aumentar y alcanzaron un F1 macro de 0.84 en PROMISE y 0.90 en PROMISE_exp. Shreda y Hanani [43] combinaron SMOTE con submuestreo y Kiani et al. [48] usaron un sobremuestreo modificado con

un ensamble de transformers. Otros estudios recurren a pesos de clase y Focal Loss [31], [53], [58] o a la descomposición del entrenamiento en subconjuntos balanceados [19].

Varios estudios evitan el problema en lugar de tratarlo: eliminan la clase con un único ejemplo [12], [62], restringen la tarea a las subclases frecuentes [14], [16], [46], agrupan las raras en una clase "otras" [21] o construyen un conjunto nuevo balanceado [15]. Donde el F1 macro está disponible, la brecha con el ponderado es amplia: BERT sin tratamiento obtiene un F1 macro de 0.34 en PROMISE y 0.44 en PROMISE_exp [45], y regresión logística, 0.66 en las subclases de NFR [58].

III-D. PR3: LLM como clasificadores

En la síntesis, 12 estudios usan un LLM como clasificador y 3 como generador de datos. Los resultados dependen del escenario. Con *few-shot* y prompts diseñados, Binkhonain y Alfayez [30] alcanzaron un F1 macro de 0.92 en la tarea FR/NFR y de 0.94 en la multiclase de NFR, con pruebas Scott-Knott ESD y tamaño del efecto de Cliff. Tang et al. [47] mostraron que añadir ejemplos puede empeorar el resultado (*over-prompting*) y que el F1 macro (0.60–0.80) queda por debajo del ponderado en todos los modelos, porque los LLM favorecen las clases dominantes. Shafikuzzaman et al. [46] obtuvieron con GPT-4o *few-shot* un F1 macro de 0.80 en las cuatro NFR frecuentes, frente a 0.66 de SBERT, aunque NoRBERT siguió siendo superior en la tarea FR/NFR; además excluyeron del conjunto de prueba los ejemplos del prompt para evitar la fuga. Con recuperación de requerimientos similares como evidencia, un LLM alcanzó un F1 macro de 0.89 en la multiclase de NFR [56]. La temperatura altera el equilibrio entre precisión y exhaustividad, y GPT-4o es menos consistente entre ejecuciones que LLaMA 3.3 [41]. En cambio, ChatGPT y Gemini sin ejemplos quedaron 18 puntos por debajo del mejor clasificador ajustado en PROMISE (0.78 frente a 0.96) [40]. Sobre issues de GitHub, el mejor de cuatro LLM sin ejemplos alcanzó un F1 macro de 0.64, con mantenibilidad entre 0.15 y 0.52 según el modelo [57]. En reseñas, ChatGPT igualó a un grupo de evaluadores legos al clasificar características de calidad (exactitud 0.66 frente a 0.63–0.72), sin superarlo, y tendió a asignar la clase mayoritaria [34]. Los clasificadores *zero-shot* basados en codificadores, como SBERT, obtienen un F1 ponderado de 0.66 en FR/NFR y de 0.72–0.80 en las NFR frecuentes, con valores más bajos en las clases raras [23].

La comparación es incompleta en todos los casos: los LLM se contrastan con transformers ajustados sin datos aumentados [30], con modelos de inferencia y de prompts [27] o con ML clásico [38]. Ningún estudio compara un LLM con un BERT entrenado con aumentación.

III-E. PR4: conjuntos y fuentes de datos

PROMISE y sus variantes dominan: 103 de los 136 estudios del mapeo usan especificaciones de requerimientos, en su mayoría PROMISE o PROMISE_exp, y 40 de los 58 de la síntesis los usan. Fuera del inglés hay conjuntos en español [60], turco [20] y árabe [18], y en formato ágil, 10,006 historias de usuario etiquetadas con características de ISO/IEC 25010 [31]. Para superar su tamaño, varios estudios combinan fuentes: PROMISE_exp con PURE y especificaciones recopiladas (3,072 requerimientos) [59], nueve fuentes públicas y privadas (4,732) [44] o cuatro conjuntos depurados de duplicados [55]. Las solicitudes de foros de proyectos de código abierto son un antecedente temprano del uso de texto de desarrolladores y usuarios [11]. Las alternativas son PURE [43], [54], SecReq, las reseñas de aplicaciones (16 estudios) [34], [38], [42], [65], los issues de GitHub (14) y las historias de usuario (3). GitReq [57] es el primer conjunto público de issues de GitHub clasificados en categorías de calidad: 6,302 requerimientos validados por expertos (Fleiss $\kappa = 0{,}72$) en ocho clases alineadas con ISO/IEC 25010, depurados de duplicados por similitud y con escalabilidad (157) y mantenibilidad (144) como clases menores. La tarea binaria FR/NFR aparece en 47 estudios y la multietiqueta en 15. Las vías para ampliar los conjuntos son el aprendizaje semisupervisado [10], el *few-shot* [29] y la generación sintética [39]. La calidad de PROMISE es una limitación reconocida: NoRBERT y DBGAT advierten que contiene duplicados y etiquetas erróneas y usan una versión reetiquetada [3], [14].

Tabla X
RESULTADO DE LA EVALUACIÓN DE CALIDAD ($n = 58$)

Criterio	Sí	Parcial	No	Media
QA1 Objetivos	58	0	0	1.00
QA2 Datos descritos	50	8	0	0.93
QA3 Protocolo sin fuga	19	31	8	0.59
QA4 Métricas para desbalance	19	33	6	0.61
QA5 Línea base	44	14	0	0.88
QA6 Variabilidad o pruebas	18	7	33	0.37
QA7 Reproducibilidad	17	14	27	0.41
QA8 Limitaciones	41	9	8	0.78

III-F. PR5: rigor de la evaluación

La Tabla X resume la evaluación de calidad. Los criterios más débiles son la variabilidad o las pruebas estadísticas (QA6) y la reproducibilidad (QA7): 19 de 58 estudios aplican pruebas de significancia y 17 publican código o datos. Le siguen el protocolo sin fuga (QA3) y las métricas adecuadas para el desbalance (QA4): 17 estudios reportan F1 macro y los demás informan exactitud, F1 ponderado o micro, que ocultan el desempeño en las clases raras. Binkhonain y Zhao [19] señalaron que NoRBERT reporta F1 ponderado, sesgado hacia las clases mayoritarias.

La fuga de datos es el riesgo principal para comparar técnicas de aumentación. Solo 10 estudios controlan los duplicados entre entrenamiento y prueba, 10 no lo hacen y 31 no lo indican. De los estudios que remuestrean o aumentan, 8 lo hacen solo en el entrenamiento, 6 no lo precisan y 5 remuestrean o aumentan todo el conjunto antes de partirlo [17], [25], [26], [35], [59]. En ese caso, las copias o variantes de un requerimiento de prueba pueden quedar en entrenamiento; uno de ellos reporta exactitudes superiores a 0.98, coherentes con

esa fuga [26]. Nguyen Van y Ahmed [55] señalan además que la validación cruzada aleatoria domina la literatura y puede dar estimaciones optimistas. Dos estudios lo cuantifican: el mismo modelo obtuvo un F1 macro de 0.97 con validación cruzada por oración y de 0.80 en proyectos no vistos [53], y SBERT pasó de un F1 de 0.89 en validación cruzada a 0.71 en reseñas nuevas [64]. TAOS aumenta solo el entrenamiento, pero su partición pública mezcla ambos conjuntos: al verificar su repositorio, 240 de los 461 requerimientos de prueba (52 %) también figuran en entrenamiento, y sus 1,534 filas corresponden a 988 requerimientos distintos.

IV. Discusión

La evidencia confirma las brechas del estado del arte con más precisión. Primero, las técnicas contra el desbalance se evalúan casi siempre aisladas: ningún estudio compara TAOS, EDA y SMOTE-Tomek con el mismo clasificador y la misma partición, y las dos comparaciones de varias aumentaciones se limitan a la tarea binaria de seguridad [37] o a un BiGRU [50]. Ambas muestran que el efecto depende de la técnica y del modelo, lo que refuerza la necesidad de una comparación controlada con BERT. Segundo, los LLM se comparan con transformers sin aumentación, por lo que no se sabe si un BERT entrenado con datos aumentados los iguala o supera. Tercero, la ganancia de la aumentación es incierta: la mayor mejora reportada, la de TAOS, se midió sobre una partición con fuga; la aumentación contextual con BERT se aplicó antes de partir el conjunto [59], y la generación con GPT-4o no superó a los datos reales en PROMISE.

A estas brechas temáticas se suma una metodológica. Con pocos estudios que reportan F1 macro, controlan duplicados o aplican pruebas estadísticas, las cifras publicadas no son comparables entre sí. Para la investigación propuesta se derivan cuatro requisitos: eliminar los duplicados antes de partir, aumentar solo el entrenamiento dentro de cada pliegue de una validación cruzada estratificada, reportar F1 macro y por clase, y contrastar las diferencias con pruebas estadísticas. Los estudios mejor evaluados [3], [19], [23], [30], [46], [51], [55] sirven de referencia para estas prácticas.

V. Amenazas a la validez

Cobertura de la búsqueda. La búsqueda usó una sola base. Scopus indexa las editoriales donde se publica el área, pero pueden faltar trabajos indexados solo en otras bases o en repositorios de preprints. Para mitigarlo, se combinaron cuatro ejes temáticos con sinónimos, variantes ortográficas y operadores de proximidad, y los trabajos base del periodo más citados por los incluidos, como Kurtanović y Maalej y NoRBERT, figuran en el corpus.

Sesgo de selección. El cribado inicial se realizó con apoyo de un modelo de lenguaje. Para controlarlo, dos autores validaron de forma independiente y ciega una muestra de las decisiones, se midió el acuerdo con kappa y se documentó el criterio aplicado a cada registro.

Disponibilidad del texto completo. La síntesis abarca los 58 estudios cuyo texto completo se obtuvo; los 78 restantes solo contribuyen al mapeo. Los estudios de acceso abierto podrían diferir de los de pago, y en tres casos se leyó el preprint de arXiv, que puede diferir en detalles de la versión publicada. Esta limitación se reduce a medida que se incorporan los textos completos obtenidos por la suscripción institucional.

Validez de la extracción. La extracción la realizó un revisor con apoyo de un modelo de lenguaje y verificando cada cifra en el texto del estudio; dos autores revisan una muestra. Las etiquetas del mapeo se asignaron a partir de los resúmenes.

Validez de las conclusiones. La heterogeneidad de conjuntos, clases y métricas impide comparar cifras entre estudios; por ello, la síntesis es narrativa y se pondera por calidad.

VI. Conclusiones

PR1. La clasificación de NFR pasó del ML clásico a los transformers y, desde 2024, a los LLM; los transformers ajustados siguen siendo la referencia en la tarea multiclase. **PR2.** El desbalance se reconoce pero rara vez se trata; casi todos los estudios evalúan una sola técnica, varios evitan las clases raras y las pocas comparaciones entre aumentaciones muestran que el efecto depende de la técnica y del modelo. **PR3.** Los LLM con *few-shot* compiten con BERT, pero favorecen las clases dominantes y nunca se comparan con un BERT aumentado. **PR4.** PROMISE y sus variantes dominan pese a sus duplicados; PURE, las reseñas y GitHub son alternativas emergentes. **PR5.** El rigor es bajo en las pruebas estadísticas, la reproducibilidad y el control de la fuga de datos. La comparación controlada de TAOS, EDA y SMOTE-Tomek con BERT, contrastada con LLM sobre un conjunto sin duplicados y con métricas por clase, ocupa un espacio que la literatura revisada deja abierto.

Disponibilidad de datos. Las cadenas de búsqueda, las decisiones de cribado con el criterio aplicado, el formulario de extracción y evaluación de calidad, y el código del análisis están disponibles en el repositorio del proyecto (https://github.com/AlessandroPastor/research).

Referencias

[1] J. Cleland-Huang, R. Settimi, X. Zou, and P. Solc, "Automated classification of non-functional requirements," *Requirements Engineering*, vol. 12, no. 2, pp. 103–120, 2007.

[2] Z. Kurtanović and W. Maalej, "Automatically classifying functional and non-functional requirements using supervised machine learning," in *2017 IEEE 25th International Requirements Engineering Conference (RE)*, 2017, pp. 490–495.

[3] T. Hey, J. Keim, A. Koziolek *et al.*, "NoRBERT: Transfer learning for requirements classification," in *2020 IEEE 28th International Requirements Engineering Conference (RE)*, 2020, pp. 169–179.

[4] B. Ben Ammar and N. F. Almatrafi, "Classification of requirements from software requirement specification documents: TAOS: A novel TextAttack oversampling approach for fine-grained classification of software requirements," *Engineering, Technology & Applied Science Research*, vol. 15, no. 6, pp. 29 872–29 879, 2025.

[5] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Keele University and Durham University, Tech. Rep. EBSE-2007-01, 2007.

[6] B. A. Kitchenham, D. Budgen, and P. Brereton, *Evidence-Based Software Engineering and Systematic Reviews*. Chapman and Hall/CRC, 2015.

[7] M. J. Page, J. E. McKenzie, P. M. Bossuyt *et al.*, "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," *BMJ*, vol. 372, p. n71, 2021.

[8] A. El-Hajjami and C. Salinesi, "Synthline: A product line approach for synthetic requirements engineering data generation using large language models," in *Lecture Notes in Business Information Processing*, 2025, pp. 208–225.

[9] T. Dybå and T. Dingsøyr, "Empirical studies of agile software development: A systematic review," *Information and Software Technology*, vol. 50, no. 9–10, pp. 833–859, 2008.

[10] R. Deocadez, R. Harrison, and D. Rodriguez, "Automatically classifying requirements from app stores: A preliminary study," in *2017 IEEE 25th International Requirements Engineering Conference Workshops (REW)*, 2017, pp. 367–371.

[11] C. Li, L. Huang, J. Ge, B. Luo, and V. Ng, "Automatically classifying user requests in crowdsourcing requirements engineering," *Journal of Systems and Software*, vol. 138, pp. 108–123, 2018.

[12] L. Tóth and L. Vidács, "Study of various classifiers for identification and classification of non-functional requirements," in *Lecture Notes in Computer Science*, 2018, pp. 492–503.

[13] M. Younas, K. Wakil, D. N., M. Arif, and A. Mustafa, "An automated approach for identification of non-functional requirements using word2vec model," *International Journal of Advanced Computer Science and Applications*, vol. 10, no. 8, 2019.

[14] G. Li, C. Zheng, M. Li, and H. Wang, "Automatic requirements classification based on graph attention network," *IEEE Access*, vol. 10, pp. 30 080–30 090, 2022.

[15] B. Li and X. Nong, "Automatically classifying non-functional requirements using deep neural network," *Pattern Recognition*, vol. 132, p. 108948, 2022.

[16] K. Kaur and P. Kaur, "Bert-cnn: Improving bert for requirements classification using cnn," in *Procedia Computer Science*, vol. 218, 2022, pp. 2604–2611.

[17] S. Vijayvargiya, L. Kumar, L. B. Murthy, and S. Misra, "Software requirements classification using deep-learning approach with various hidden layers," in *Annals of Computer Science and Information Systems*, vol. 30, 2022, pp. 895–904.

[18] A. E. Yahya, A. Gharbi, W. M. S. Yafooz, and A. Al-Dhaqm, "A novel hybrid deep learning model for detecting and classifying non-functional requirements of mobile apps issues," *Electronics (Switzerland)*, vol. 12, no. 5, p. 1258, 2023.

[19] M. Binkhonain and L. Zhao, "A machine learning approach for hierarchical classification of software requirements," *Machine Learning with Applications*, vol. 12, p. 100457, 2023.

[20] F. Yucalar, "Developing an advanced software requirements classification model using bert: An empirical evaluation study on newly generated turkish data," *Applied Sciences (Switzerland)*, vol. 13, no. 20, p. 11127, 2023.

[21] K. Kaur and P. Kaur, "Improving bert model for requirements classification by bidirectional lstm-cnn deep model," *Computers and Electrical Engineering*, vol. 108, p. 108699, 2023.

[22] M. A. Khan, M. S. Khan, I. Khan, S. Ahmad, and S. Huda, "Non functional requirements identification and classification using transfer learning model," *IEEE Access*, vol. 11, pp. 74 997–75 005, 2023.

[23] W. Alhoshan, A. Ferrari, and L. Zhao, "Zero-shot learning for requirements classification: An exploratory study," *Information and Software Technology*, vol. 159, p. 107202, 2023.

[24] T. Bagies, "Classifying software security requirements into confidentiality, integrity, and availability using machine learning approaches," *PeerJ Computer Science*, vol. 10, pp. 1–20, 2024.

[25] J. Abbas, C. Zhang, and B. Luo, "Enscl-catboost: A strategic framework for software requirements classification," *IEEE Access*, vol. 12, pp. 127 614–127 628, 2024.

[26] M. A. Metu, N. Akhter, S. Nasrin, T. Anzum, A. Khatun, and R. Mazumder, "Hybrid svm-bidirectional long short-term memory model for fine-grained software requirement classification," *Journal of Advances in Information Technology*, vol. 15, no. 8, pp. 914–922, 2024.

[27] M. Fazelnia, V. Koscinski, S. Herzog, and M. Mirakhorli, "Lessons from the use of natural language inference (nli) in requirements engineering tasks," in *Proceedings of the IEEE International Conference on Requirements Engineering*, 2024, pp. 103–115.

[28] S. Saleem, M. N. Asim, L. Van Elst, M. Junker, and A. Dengel, "Mlr-predictor: a versatile and efficient computational framework for multi-label requirements classification," *Frontiers in Artificial Intelligence*, vol. 7, p. 1481581, 2024.

[29] M. B. Saleem, N. Q. M. Noor, U. Omer, R. Danial, and K. Sardar, "Analyzing the impact of machine learning algorithms on software requirements classification," *VFAST Transactions on Software Engineering*, vol. 13, no. 1, pp. 88–98, 2025.

[30] M. Binkhonain and R. Alfayez, "Are prompts all you need? Evaluating prompt-based large language models (LLM)s for software requirements classification," *Requirements Engineering*, vol. 30, no. 4, pp. 423–443, 2025.

[31] A. Alhaizaey and M. Al-Mashari, "Automated classification and identification of non-functional requirements in agile-based requirements using pre-trained language models," *IEEE Access*, vol. 13, pp. 87 401–87 417, 2025.

[32] S. Eltahier, O. Dawood, and I. Saeed, "Bert fine-tuning for software requirement classification: Impact of model components and dataset size," *Information (Switzerland)*, vol. 16, no. 11, p. 981, 2025.

[33] F. Casillo, V. Deufemia, and C. Gravino, "Beyond domain dependency in security requirements identification," *Information and Software Technology*, vol. 182, p. 107702, 2025.

[34] E. C. Groen, F. Dalpiaz, M. van Vliet, B. Winter, J. Doerr, and S. Brinkkemper, "Classification of quality characteristics in online user feedback using linguistic analysis, crowdsourcing and llms," *Journal of Systems and Software*, vol. 230, p. 112533, 2025.

[35] H. Ali, U. Tanveer, A. Saeed, H. K. Alkahtani, K. J. Alzahrani, and B. Akbayan, "Cloud-based machine learning for scalable classification of software requirements: Insights from the promise dataset," *Systems and Soft Computing*, vol. 7, p. 200405, 2025.

[36] A. F. Subahi, "Enhancing software sustainability: Leveraging large language models to evaluate security requirements fulfillment in requirements engineering," *Systems*, vol. 13, no. 2, p. 114, 2025.

[37] L. Petrillo, F. Martinelli, A. Santone, and F. Mercaldo, "Explainable security requirements classification through transformer models," *Future Internet*, vol. 17, no. 1, p. 15, 2025.

[38] M. Chaudhary, C. Jain, and P. R. Anish, "Exploring zero-shot app review classification with chatgpt: Challenges and potential," in *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering , EASE, 2025 edition, EASE 2025*, 2025, pp. 672–677.

[39] T. K. B. Pham, A. Alshomar, S. Supakkul, T. Hill, and L. Chung, "Generating synthetic nonfunctional requirements using large language models for training machine learning classifiers (s)," in *International Conferences on Software Engineering and Knowledge Engineering*, vol. 2025, 2025, pp. 217–222.

[40] S. Saleem, M. N. Asim, L. V. Elst, and A. Dengel, "Generative language models potential for requirement engineering applications: insights into current strengths and limitations," *Complex and Intelligent Systems*, vol. 11, no. 6, p. 278, 2025.

[41] F. Karlsson, P. Chatzipetrou, S. Gao, and T. E. Havstorm, "How reliable are gpt-4o and llama3.3-70b in classifying natural language requirements?: The impact of the temperature setting," *IEEE Software*, vol. 42, no. 6, pp. 97–104, 2025.

[42] N. Alsaleh, R. Alnanih, and N. Alowidi, "Hybrid deep learning approach for automating app review classification: Advancing usability metrics classification with an aspect-based sentiment analysis framework," *Computers, Materials and Continua*, vol. 82, no. 1, pp. 949–976, 2025.

[43] Q. A. Shreda and A. A. Hanani, "Identifying non-functional requirements from unconstrained documents using natural language processing and machine learning approaches," *IEEE Access*, vol. 13, pp. 124 159–124 179, 2025.

[44] T. Alsanoosy, "Large language model for requirements classification: An ensemble approach," in *Procedia Computer Science*, vol. 270, 2025, pp. 3648–3657.

[45] K. Aryan and U. Shah, "Multi-level software requirements classification: Assessing bert variants and their hybrid models," in *Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE*, 2025, pp. 330–335.

[46] M. Shafikuzzaman, M. R. Islam, S. Zaman, A. Ma, and A. Islam Sifat, "On the effectiveness of zero-shot and few-shot pretrained language models for software requirement classification," *IEEE Access*, vol. 13, pp. 159 439–159 453, 2025.

[47] Y. Tang, D. Tuncel, C. Koerner, and T. Runkler, "The few-shot dilemma: Over-prompting large language models," in *2025 3rd International*

Conference on Foundation and Large Language Models (FLLM), 2025, pp. 134–141.

[48] A. A. Kiani, "A deep learning approach to classifying software requirements: The application of transformer-based ensemble learning and attention-based fusion," *Foundations of Computing and Decision Sciences*, vol. 51, no. 1, pp. 71–110, 2026.

[49] A. Bhar, S. Sengupta, and S. Choudhury, "A knowledge-driven semantic framework for non-functional requirement engineering in intelligent healthcare systems," *Discover Artificial Intelligence*, vol. 6, no. 1, p. 446, 2026.

[50] O. Q. Aldhafer, H. A. Alsadiq, M. Alshaboti *et al.*, "Adaptive data augmentation techniques for software requirements classification using deep learning," *IEEE Access*, vol. 14, pp. 43 841–43 854, 2026.

[51] M. G. Said, L. M. El-Fangary, S. M. Azzam, and A. A. Abohany, "An intelligent model to predict functional and non-functional requirements from software requirements specification," *IEEE Access*, vol. 14, pp. 40 497–40 511, 2026.

[52] L. Petrillo, F. Mercaldo, and A. Santone, "An explainable method for security requirements classification through transformer models," *Future Generation Computer Systems*, vol. 183, p. 108544, 2026.

[53] A. Almohammady, R. Alnanih, and N. Alowidi, "Context-aware non-functional requirement classification using functional requirement guidance and domain-specialized representations," *Electronics (Switzerland)*, vol. 15, no. 18, p. 4299, 2026.

[54] F. Khayashi, B. Jamasb, R. Akbari, and P. Shamsinejadbabaki, "Deep learning methods for software requirement classification: A performance study on the pure dataset," *International Journal of Computational Intelligence and Applications*, p. 2650037, 2026.

[55] N. Nguyen Van and M. Ahmed, "Evaluating robustness of hierarchical and multi-label requirements classification: Effects of data partitioning and supervision design," *Information and Software Technology*, vol. 198, p. 108212, 2026.

[56] N. Aloufi and A. Aljuhani, "Evireqrag: An evidence-based retrieval-augmented generation framework for software requirements and security classification," *IEEE Access*, vol. 14, pp. 146 435–146 471, 2026.

[57] F. Kamal, M. H. Kabir, and M. R. Islam, "Gitreq: A gold standard dataset for software quality requirements," in *2026 IEEE/ACIS 24th International Conference on Software Engineering Research, Management and Applications (SERA)*, 2026, pp. 116–121.

[58] R. Nagaraju and V. Anantha Natarajan, "Hierarchical software requirement classification using transformer-based language models," *International Journal of Artificial Intelligence and Machine Learning*, vol. 6, no. 6s, pp. 340–359, 2026.

[59] J. Kaur and C. Roy, "Hybrid approach for multilevel multi-class requirement classification: Impact of stop-word removal and data augmentation," *Journal of Systems and Software*, vol. 231, p. 112594, 2026.

[60] M. Limaylla-Lunarejo, N. Condori-Fernandez, M. Rodríguez Luaces, and O. Karras, "Improving the multi-class classification of non-functional requirements in Spanish: A study of dataset balancing and performance," *Empirical Software Engineering*, vol. 31, 2026.

[61] G. Bishop, S. Chakraborty, H. Zhou, J. Dehlinger, L. Deng, J. Lin, and B. Kist, "On-demand generation of high-quality software engineering datasets using large language models and ontologies," *Automated Software Engineering*, vol. 33, no. 3, p. 73, 2026.

[62] K. Rahman, A. Ghani, A. U. Rahman, S. Misra, and D. H. Kim, "Requibert: Transfer learning for software requirements classifications," *Expert Systems with Applications*, vol. 327, p. 132726, 2026.

[63] W. Alhoshan, "Seqfal: A federated active learning framework for private and efficient labeling of security requirements," *Applied Sciences*, vol. 16, no. 2, p. 914, 2026.

[64] T. Bagies, "Transformer-based classification of security-related mobile app reviews using active learning," *IEEE Access*, vol. 14, pp. 72 918–72 942, 2026.

[65] C. Wellhausen, L. Reinhardt, and K. Schneider, "User reviews as a source for usability requirements: A precursor study on using large language models," in *CEUR Workshop Proceedings*, vol. 4208, 2026.

APÉNDICE

ANEXO A. CADENAS DE BÚSQUEDA COMPLETAS

Las cuatro cadenas se ejecutaron en Scopus (*Advanced document search*) en octubre de 2026. Los registros de las cuatro se unieron y se eliminaron los duplicados entre cadenas.

S1: aumentación y desbalance.

```
TITLE-ABS-KEY ( ( ( "non-functional" OR "non functional" OR nonfunctional OR quality OR NFR OR NFRs ) W/3 requirement* )
OR "software requirement*" OR "requirements specification*" OR "requirement* classification" OR "PROMISE NFR" OR
"PROMISE_exp" ) AND TITLE-ABS-KEY ( classif* OR categori* ) AND TITLE-ABS-KEY ( "data augmentation" OR "text augmentation"
OR "easy data augmentation" OR TextAttack OR TAOS OR "synthetic data" OR "synthetic sample*" OR oversampl* OR
"over-sampling" OR resampl* OR SMOTE OR Tomek OR "class imbalance" OR "imbalanced data*" OR "imbalanced class*" OR
"minority class*" OR "back translation" OR paraphras* ) AND PUBYEAR > 2016 AND PUBYEAR < 2027 AND ( LIMIT-TO (
DOCTYPE,"ar" ) OR LIMIT-TO ( DOCTYPE,"cp" ) OR LIMIT-TO ( DOCTYPE,"re" ) OR LIMIT-TO ( DOCTYPE,"ch" ) ) AND ( LIMIT-TO (
LANGUAGE,"English" ) OR LIMIT-TO ( LANGUAGE,"Spanish" ) )
```

S2: BERT y transformers.

```
TITLE-ABS-KEY ( ( ( "non-functional" OR "non functional" OR nonfunctional OR quality OR NFR OR NFRs ) W/3 requirement* )
OR "software requirement*" OR "requirements specification*" OR "requirement* classification" OR "PROMISE NFR" OR
"PROMISE_exp" ) AND TITLE-ABS-KEY ( classif* OR categori* ) AND TITLE-ABS-KEY ( BERT OR RoBERTa OR DistilBERT OR NoRBERT
OR transformer* OR "pre-trained language model*" OR "pretrained language model*" OR "transfer learning" OR "fine tun*" )
AND PUBYEAR > 2016 AND PUBYEAR < 2027 AND ( LIMIT-TO ( DOCTYPE,"ar" ) OR LIMIT-TO ( DOCTYPE,"cp" ) OR LIMIT-TO (
DOCTYPE,"re" ) OR LIMIT-TO ( DOCTYPE,"ch" ) ) AND ( LIMIT-TO ( LANGUAGE,"English" ) OR LIMIT-TO ( LANGUAGE,"Spanish" ) )
```

S3: LLM como clasificador.

```
TITLE-ABS-KEY ( ( ( "non-functional" OR "non functional" OR nonfunctional OR quality OR NFR OR NFRs ) W/3 requirement* )
OR "software requirement*" OR "requirements specification*" OR "requirement* classification" OR "PROMISE NFR" OR
"PROMISE_exp" ) AND TITLE-ABS-KEY ( classif* OR categori* ) AND TITLE-ABS-KEY ( "large language model*" OR LLM OR LLMs OR
GPT* OR ChatGPT OR "generative AI" OR Llama OR Gemma OR Mistral OR Claude OR "zero-shot" OR "few-shot" OR "in-context
learning" OR prompt* ) AND PUBYEAR > 2016 AND PUBYEAR < 2027 AND ( LIMIT-TO ( DOCTYPE,"ar" ) OR LIMIT-TO ( DOCTYPE,"cp" )
OR LIMIT-TO ( DOCTYPE,"re" ) OR LIMIT-TO ( DOCTYPE,"ch" ) ) AND ( LIMIT-TO ( LANGUAGE,"English" ) OR LIMIT-TO (
LANGUAGE,"Spanish" ) )
```

S4: GitHub y auto-etiquetado.

```
TITLE-ABS-KEY ( ( ( "non-functional" OR "non functional" OR nonfunctional OR quality OR NFR OR NFRs ) W/3 requirement* )
OR "software requirement*" OR "requirements specification*" OR "requirement* classification" OR "PROMISE NFR" OR
"PROMISE_exp" ) AND TITLE-ABS-KEY ( classif* OR categori* ) AND TITLE-ABS-KEY ( GitHub OR "issue tracker*" OR "issue
report*" OR "open source" OR "open-source" OR "semi-supervised" OR "self-training" OR "pseudo-label*" OR "active learning"
OR "weak supervision" OR "dataset construction" OR "gold standard" ) AND PUBYEAR > 2016 AND PUBYEAR < 2027 AND ( LIMIT-TO
( DOCTYPE,"ar" ) OR LIMIT-TO ( DOCTYPE,"cp" ) OR LIMIT-TO ( DOCTYPE,"re" ) OR LIMIT-TO ( DOCTYPE,"ch" ) ) AND ( LIMIT-TO (
LANGUAGE,"English" ) OR LIMIT-TO ( LANGUAGE,"Spanish" ) )
```